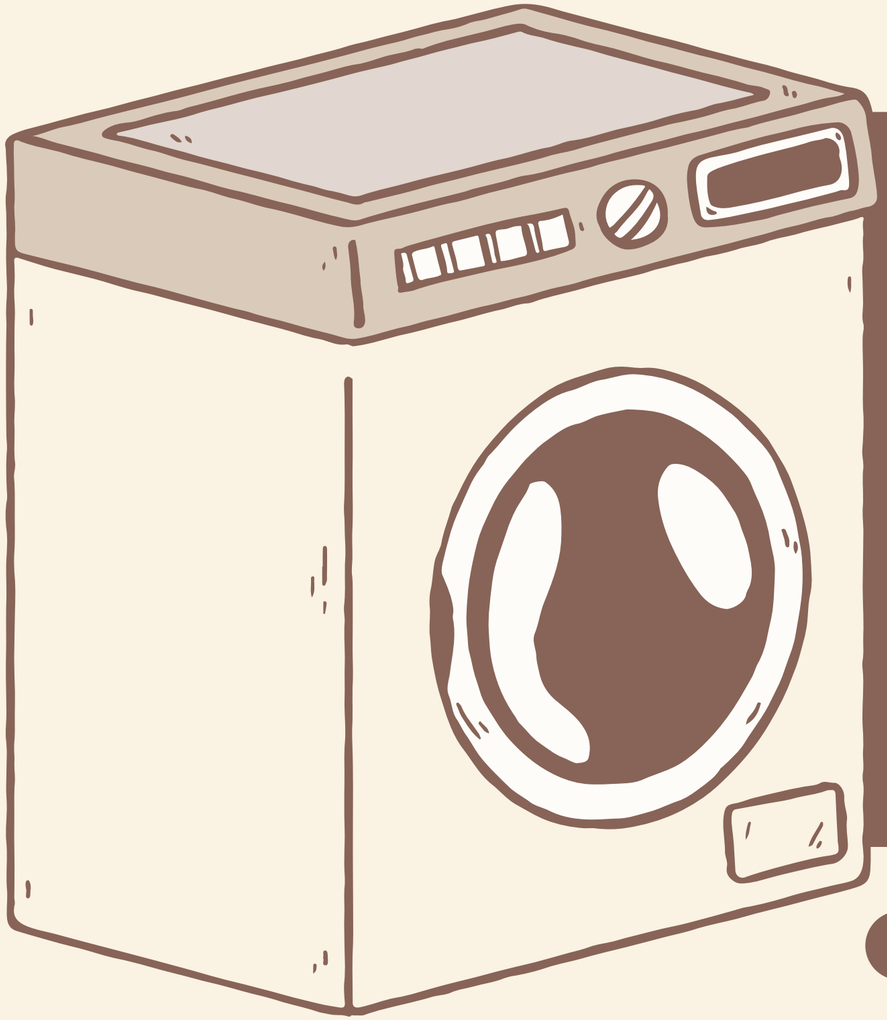
CP353002 · Sec 3



LaundryHub

ระบบจัดการร้านซักรีด · โอ๊ค พีช ปอนด์ โชกุน

Spring Boot 3.5 · Java 17

PostgreSQL · Docker · Render

LaundryHub คืออะไร?

ระบบจัดการร้านซักรีดที่รวม 2 บริการไว้ในระบบเดียว รองรับ 3 บทบาท คือ ลูกค้า (CUSTOMER) พนักงาน (STAFF) และผู้ดูแลระบบ (ADMIN) พร้อมระบบชำระเงินและแจ้งเตือนในระบบ

ฝากซัก (Full-Service)
ฝากผ้าให้ร้านซัก คิดราคาจากน้ำหนัก × ประเภทบริการ และติดตามสถานะได้



ซักเอง (Self-Service)
จองเครื่องซัก-อบเอง คิดราคาจากเวลาที่ใช้ และกันจองเวลาซ้อนกัน



สมาชิกกลุ่ม

โอ๊ค – Foundation / Security / Deploy

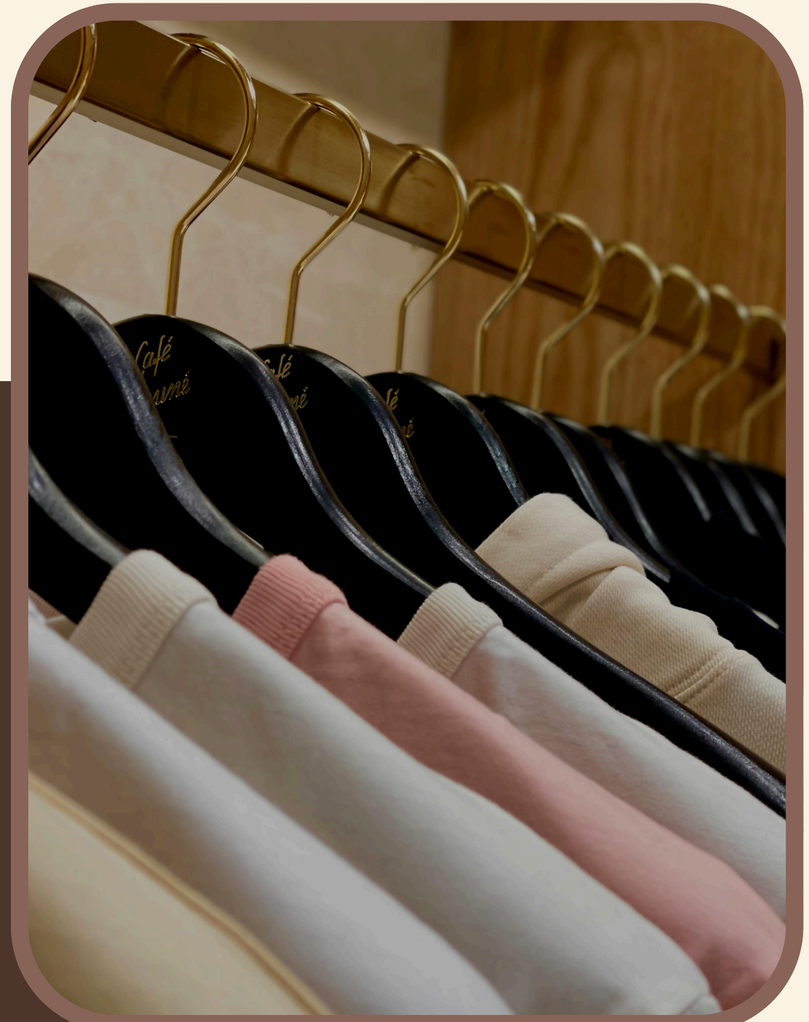
พีช – Full-Service Order (ฝากซัก)

ปอนด์ – Self-Service Machine (ซักเอง)

โชกุน – Payment / Notification / Report

เราทำงานกันอย่างไร

ทุกคนมี branch ของตัวเอง ทุกงานต้องเปิด Pull Request เข้า develop ให้เพื่อนรีวิวและ approve อย่างน้อย 1 คน และ CI (GitHub Actions) ต้องผ่าน mvn test กับ Docker build ก่อน merge



ปัญหาของร้านซักรีด

ร้านจดออเดอร์ด้วยกระดาษ ลูกค้าไม่รู้ว่าผ้าถึงขั้นไหนแล้ว เครื่องหยอดเหรียญถูกจองเวลาซ้อนกัน และเช็กยอดเงินได้ยาก LaundryHub จึงรวมทุกอย่างไว้ในระบบเดียว

ไปดูกัน



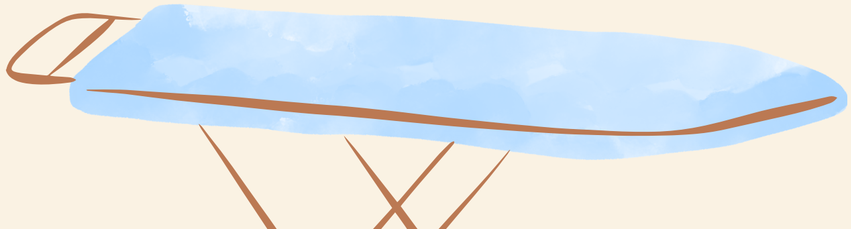
บริการหลัก

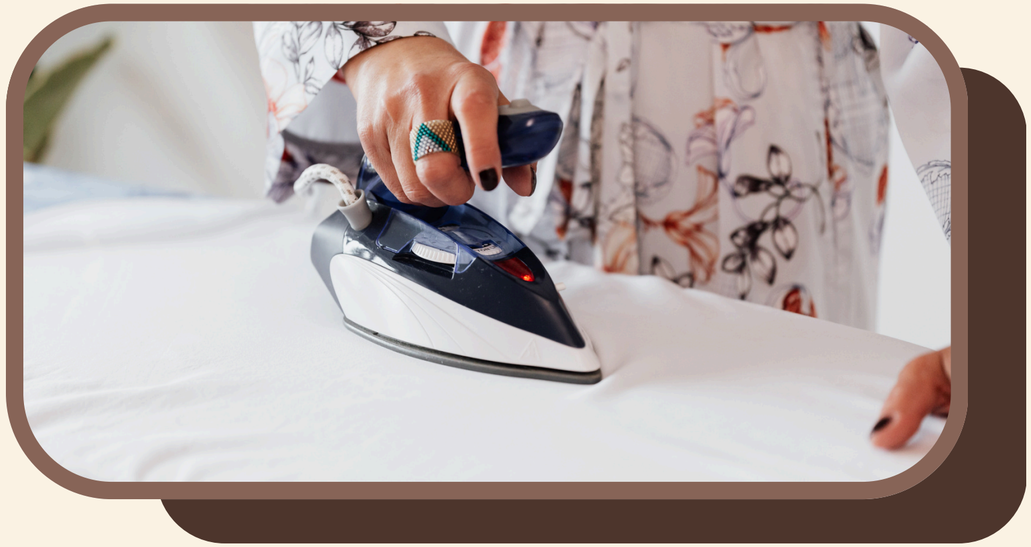


ลูกค้าเลือกได้ว่าจะฝากซักหรือซักเอง ทั้งสองแบบไปจ่ายเงินที่หน้าชำระเงินเดียวกัน และได้รับแจ้งเตือนทุกครั้งที่สถานะเปลี่ยน

ฝากซัก – พีช
สร้างออเดอร์หลายรายการ คิดราคาด้วย Strategy คุมสถานะ 7 ขั้นด้วย State Pattern พนักงานเลื่อนสถานะผ่านบอร์ด

ซักเอง – ปอนด์
ดูเครื่องว่างแล้วจองเวลา BookingValidator กันเวลาซ้อน คุมสถานะเครื่องด้วย State Pattern





ฟีเจอร์เสริม

ระบบส่วนกลางที่ทุกโมดูลใช้ร่วมกัน

ทำให้ระบบใช้งานได้ครบ ทั้งฝั่งลูกค้าและฝั่งร้าน

- ชำระเงิน: เงินสด / QR จำลอง / เหรียญ
- แจ้งเตือนในระบบอัตโนมัติ
- ล็อกอิน 3 บทบาท + รหัสผ่าน BCrypt
- REST API + Swagger UI ทดสอบได้ทันที

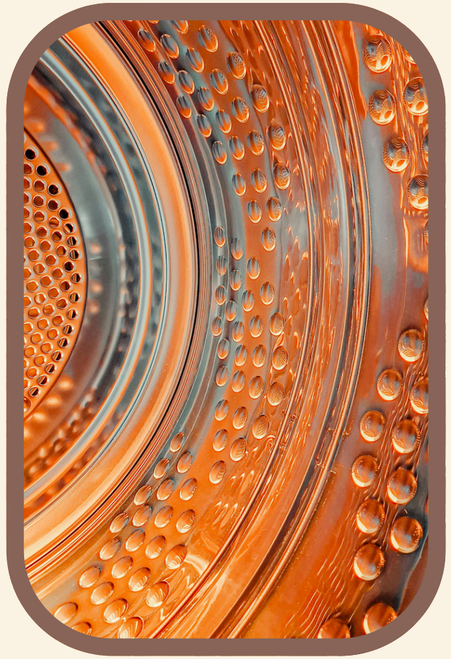


เทคโนโลยีที่ใช้

Java 17 + Spring Boot 3.5.7 build ด้วย Maven แบ่งชั้น Controller → Service → Repository



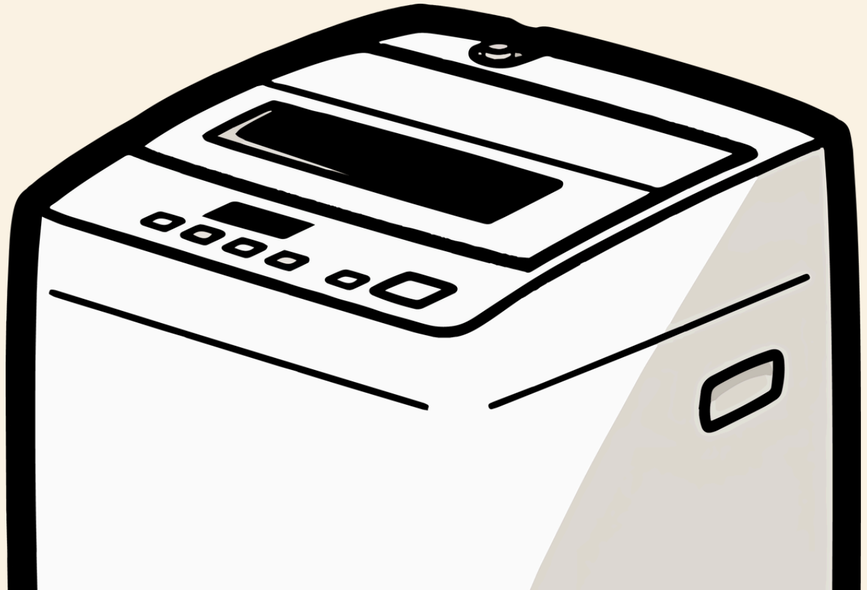
PostgreSQL 16 + Spring Data JPA ใช้ Flyway จัดการ schema



Thymeleaf + Bootstrap 5 ทำหน้าเว็บ และ Spring Security คุมสิทธิ์



Docker + GitHub Actions CI และ deploy บน Render + Neon





การทดสอบ

เทสต์กว่า 370 ตัว ด้วย JUnit 5, Mockito และ Spring Boot Test ครอบคลุม Service, Security (401/403/404) และ Web Controller ผ่านทั้งหมด

CI รัน mvn test ทุก Pull Request ถ้าไม่ผ่านจะ merge ไม่ได้



Design Patterns

ใช้ Layered Architecture, Repository, Service Layer, DTO + Mapper และ Dependency Injection เป็นโครงหลัก แล้วเลือก GoF Pattern ให้เหมาะกับแต่ละปัญหา



State
คุมสถานะออเดอร์ 7 ขั้น และสถานะเครื่อง 4 แบบ



Strategy
คิดราคาฝากซักกับซักเอง คนละสูตร และเลือกวิธีจ่ายเงิน



Observer
ยิง event เมื่อสถานะเปลี่ยน แล้วสร้างแจ้งเตือน



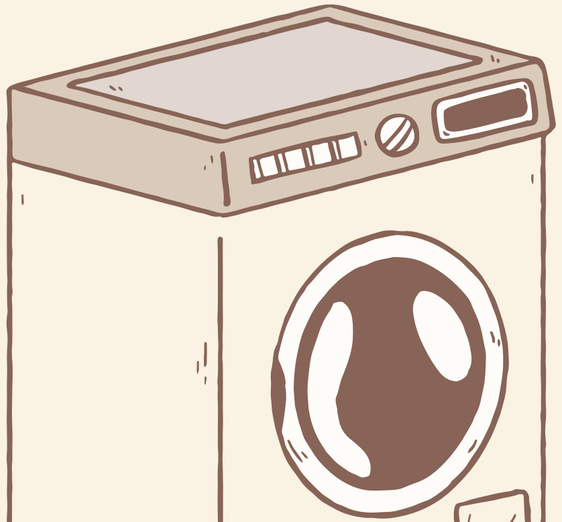
Factory + Facade + Builder
สร้างตัวจ่ายเงิน รวมขั้นตอน checkout และสร้าง DTO

Deploy & Demo

merge develop → main แล้ว Render deploy ให้อัตโนมัติ

แอปรันใน Docker deploy ขึ้น Render และใช้ฐานข้อมูล PostgreSQL บน Neon รันในเครื่องได้ด้วย docker compose คำสั่งเดียว





ลำดับเดโม: ลูกค้าสร้างออเดอร์ → พนักงานเลื่อนสถานะ → จองเครื่องซักเอง → ชำระเงิน → ได้รับแจ้งเตือน

ซอร์สโค้ด เอกสาร ไดอะแกรม และรายงานทั้งหมดอยู่ใน GitHub ของกลุ่ม ทดลองใช้งานจริงได้บน Render

ลิงก์โปรเจกต์

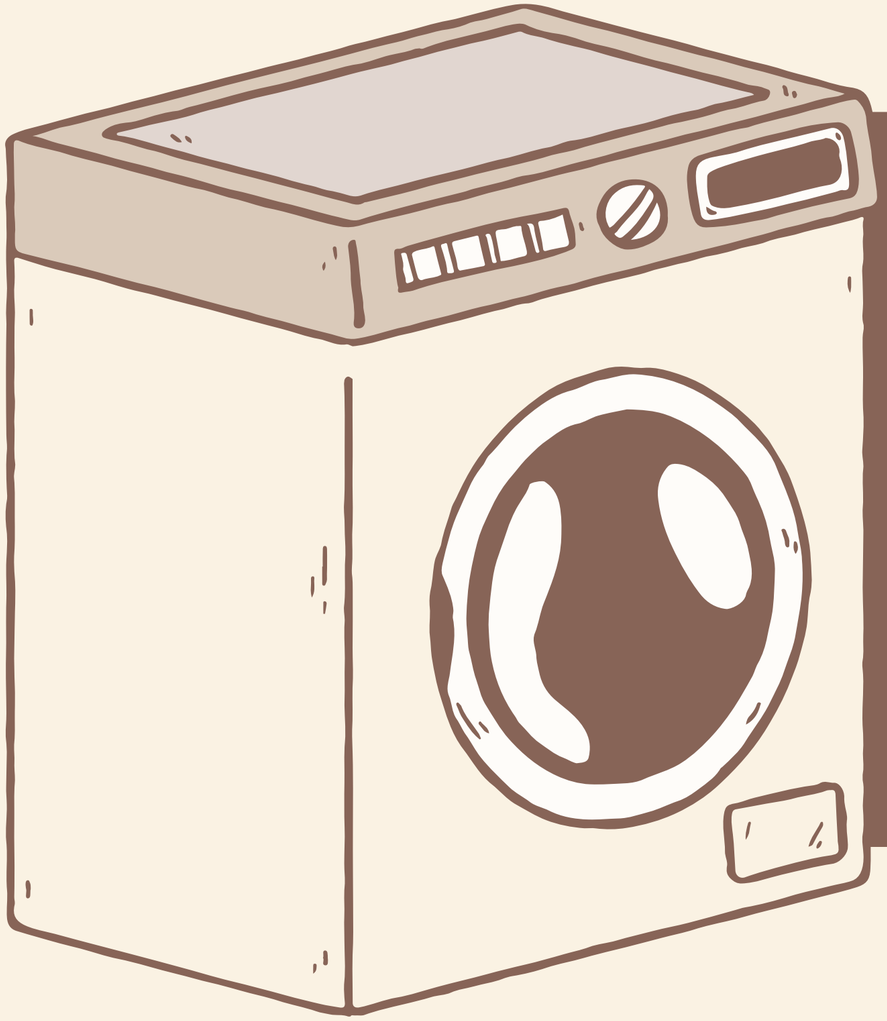
github.com/Oakiez/Laundry-Hub

laundry-hub-1ltc.onrender.com

/swagger-ui.html (API Docs)



LaundryHub



ขอบคุณครับ